

BÁO CÁO THỰC HÀNH LAB 6

SYSTEMD

Người báo cáo: Liamz07

1. Mục tiêu bài thực hành:

- Hiểu rõ cơ chế hoạt động của trình quản lý hệ thống và dịch vụ tiêu chuẩn systemd trên Linux.
- Biết cách xây dựng và kiểm tra vận hành một ứng dụng Python chạy dưới dạng tiến trình đơn giản (app.py).
- Thành thạo kỹ thuật đóng gói ứng dụng thành một dịch vụ nền hệ thống độc lập thông qua tệp cấu hình myapp.service.
- Nắm vững các thao tác điều khiển vòng đời dịch vụ bằng công cụ dòng lệnh quản trị systemctl.

2. Nội dung và phương pháp giải quyết chi tiết:

BÀI 1: TẠO VÀ KIỂM TRA ỨNG DỤNG PYTHON MẪU

- **Đề bài:** Tạo một ứng dụng Python đơn giản (app.py) làm tiến trình dịch vụ mẫu để kiểm tra khả năng ghi log liên tục.
- **Phương pháp giải quyết:** Sử dụng trình soạn thảo văn bản nano để viết mã nguồn vòng lặp in thông báo kèm khoảng thời gian chờ, sau đó kiểm tra chạy thử thủ công bằng thông dịch viên Python.
- **Lệnh thực thi & Cú pháp tổng quát:**
 - *Lệnh:* nano app.py (sau khi chỉnh sửa xong ta sử dụng python3 app.py để chạy thử)
 - *Mã nguồn ví dụ trong bài:*

```
1  import time
2  import sys
3
4  while True:
5      print("App is running...", flush=True)
6      time.sleep(5)
```

- *Cú pháp:*
 - ❖ nano [file_name]
 - ❖ python3 [file_name]
- *Giải thích:*
 - ❖ nano: Trình soạn thảo văn bản giao diện dòng lệnh trực quan trên Linux.
 - ❖ app.py: Tên tập tin mã nguồn ngôn ngữ Python chứa đoạn code thực thi vòng lặp định kỳ.
 - ❖ python3: Trình thông dịch phiên bản 3 của ngôn ngữ lập trình Python dùng để chạy trực tiếp tệp mã nguồn.

BÀI 2: BIẾN ỨNG DỤNG THÀNH SYSTEMD SERVICE

- **Đề bài:** Đóng gói ứng dụng Python thành một dịch vụ nền hệ thống với tên định danh là myapp.service.
- **Phương pháp giải quyết:** Tạo tệp cấu hình dịch vụ mới trong thư mục quản lý hệ thống /etc/systemd/system/myapp.service sử dụng quyền quản trị sudo nano, sau đó khai báo tường minh các thông số đường dẫn thực thi và cơ chế tự phục hồi lỗi.
- **Lệnh thực thi & Cú pháp tổng quát:**
 - *Lệnh:* sudo nano /etc/systemd/system/myapp.service
 - *Cú pháp:* sudo nano /etc/systemd/system/[service_name].service
 - *Nội dung cấu hình chi tiết bên trong tệp myapp.service:*

```
1  [Unit]
2  Description=My Custom Python Application Service
3  After=network.target
4
5  [Service]
6  Type=simple
7  User=root
8  WorkingDirectory=/home/kali/lab6_systemd
9  ExecStart=/usr/bin/python3 /home/kali/lab6_systemd/app.py
10 Restart=on-failure
11
12 [Install]
13 WantedBy=multi-user.target
```

- *Giải thích ý nghĩa từng thành phần trong tệp cấu hình:*
 - ❖ [Unit]: Khối chứa thông tin chung và thứ tự khởi tạo của dịch vụ.
 - ❖ Description: Mô tả ngắn gọn tên và chức năng của dịch vụ hệ thống.
 - ❖ After=network.target: Xác định dịch vụ này sẽ khởi động sau khi hệ thống đã kích hoạt xong mạng.
 - ❖ [Service]: Khối cấu hình cách thức vận hành của tiến trình dịch vụ.
 - ❖ Type=simple: Khai báo kiểu tiến trình chạy đơn giản (khi dịch vụ khởi động systemd sẽ kích hoạt ngay câu lệnh được viết ở phần ExecStart và coi đó chính là tiến trình chính để theo dõi và quản lý).
 - ❖ User=root: Chỉ định quyền tài khoản hệ thống dùng để chạy dịch vụ.
 - ❖ WorkingDirectory: Thư mục làm việc mặc định khi dịch vụ khởi chạy.
 - ❖ ExecStart: Đường dẫn tuyệt đối tới lệnh thực thi tiến trình (/usr/bin/python3 chạy kèm đường dẫn file app.py).
 - ❖ Restart=on-failure: Cấu hình tự động khởi động lại dịch vụ nếu tiến trình bị crash hoặc dừng đột ngột do lỗi.
 - ❖ [Install]: Khối cấu hình thiết lập khi dịch vụ được kích hoạt vào các chế độ chạy của hệ thống.
 - ❖ WantedBy=multi-user.target: Định nghĩa dịch vụ sẽ hoạt động ở chế độ đa người dùng (chế độ dòng lệnh tiêu chuẩn của server).

(*Ghi chú:* Một tiến trình chạy đơn giản, thường gọi là tiến trình thông thường, là một tiến trình khi được khởi chạy sẽ chiếm quyền điều khiển terminal hiện tại.)

BÀI 3: ĐIỀU KHIỂN VÀ VẬN HÀNH DỊCH VỤ VỚI SYSTEMCTL

- **Đề bài:** Sử dụng câu lệnh systemctl để cập nhật hệ thống, khởi động, theo dõi trạng thái và kiểm soát vòng đời của dịch vụ myapp.
- **Phương pháp giải quyết:** Thực hiện lệnh làm mới tiến trình systemd (daemon-reload), sau đó lần lượt kích hoạt, kiểm tra trạng thái hoạt động (status), khởi động lại (restart) và dừng dịch vụ (stop).
- **Lệnh thực thi & Cú pháp tổng quát:**
 - *Lệnh:*

```
1 sudo systemctl daemon-reload
2 sudo systemctl start myapp
3 sudo systemctl status myapp
4 sudo systemctl restart myapp
5 sudo systemctl stop myapp
```

- *Cú pháp:* sudo systemctl [command] [service_name]
- *Giải thích:*
 - ❖ systemctl: Công cụ quản lý điều khiển chính của bộ công cụ systemd.
 - ❖ daemon-reload: Lệnh yêu cầu bộ quản lý hệ thống đọc lại toàn bộ các tệp cấu hình dịch vụ mới hoặc bị thay đổi trong thư mục /etc/systemd/system/.
 - ❖ start: Kích hoạt và chạy dịch vụ nền
 - ❖ status: Hiển thị báo cáo trạng thái chi tiết của dịch vụ (bao gồm mã PID chính, mức tiêu thụ tài nguyên CPU/RAM, và các dòng log đầu ra gần nhất của ứng dụng).
 - ❖ restart: Khởi động lại dịch vụ (tương đương với việc thực hiện đồng thời lệnh stop rồi start).
 - ❖ stop: Dừng hoàn toàn hoạt động của tiến trình dịch vụ trên hệ thống.

3. Kết luận và bài học rút ra:

- **Chuyển đổi ứng dụng thành dịch vụ chuyên nghiệp:** Việc cấu hình tệp .service giúp chuyển đổi một tệp script thông thường thành một dịch vụ nền hệ thống (background service) có khả năng tự động quản lý vòng đời, tự khởi động lại khi gặp lỗi và tích hợp chặt chẽ với cơ chế quản trị của Linux.
- **Nắm vững kỹ thuật vận hành systemctl:** Quá trình thực hành củng cố vững chắc kỹ năng quản trị hệ thống, từ bước viết cấu hình, nạp lại tiến trình quản lý (daemon-reload) đến việc giám sát trạng thái thời gian thực (status), giúp đảm bảo sự ổn định cho các ứng dụng triển khai thực tế trên máy chủ.

HẾT!